

Box-and-Pointer Memory Diagrams

We visualize how Java stores primitives and Objects in memory using *box-and-pointer diagrams*, also known as memory diagrams.

Content Learning Targets

After completing this activity, you should be able to say:

- I can explain differences between primitive and reference type variables.
- I can draw box-and-pointer memory diagrams of Strings, arrays, and ArrayLists.

Process Skill Goals

During the activity, you should make progress toward:

- Justifying answers based on evidence provided. (Problem Solving)

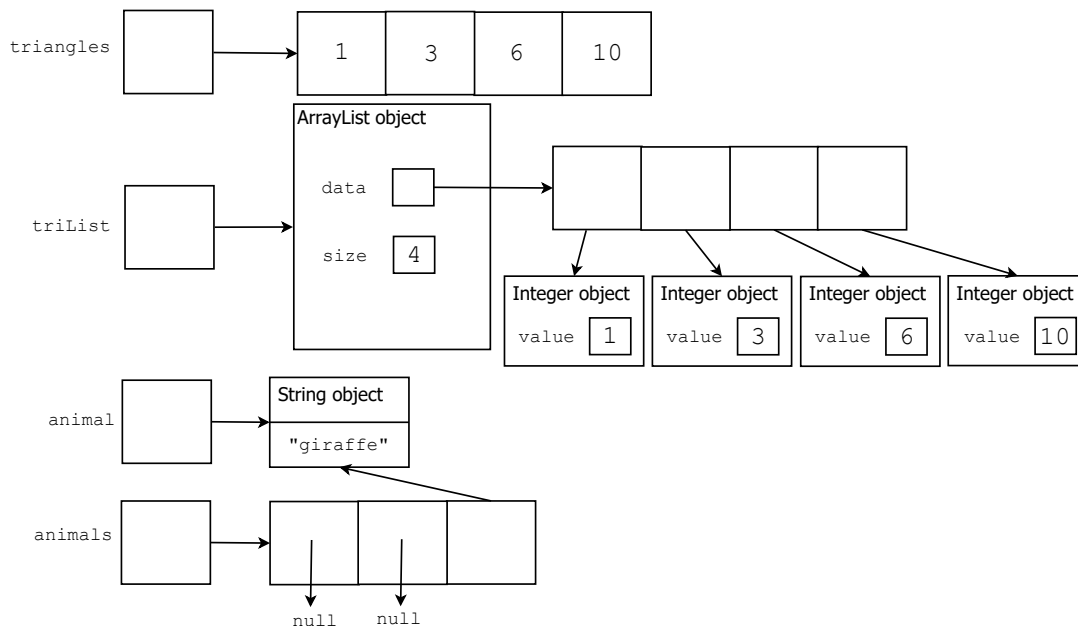


Copyright © 2026 Ian Ludden, based on [prior work](#) of Mayfield et al. This work is under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part 1 Memory Diagrams with Reference Types

In box-and-pointer memory diagrams, we use arrows for reference types (i.e., objects). Each object instance gets its own box, which contains its data (a.k.a. *fields*, or *instance variables*).

```
int[] triangles = {1, 3, 6, 10};
ArrayList<Integer> triList = new ArrayList<>();
for (Integer tri : triangles) triList.add(tri);
String animal = "giraffe";
String[] animals = new String[3];
animals[animals.length - 1] = animal;
```



Questions (15 min)

Start time:

1. What differences do you notice between the memory diagrams for `triangles` and `triList`?
2. What is the default value for reference types in arrays and `ArrayList`s?
3. Redraw to show the effects of adding `animals[1] = "tiger";` and `triList.remove(1);`.

4. (Optional) The above step should leave you with an Integer object box floating off on its own. Memory is still allocated for this Integer, but there are no longer any references to it! Java will eventually clean this up during *garbage collection*. Search the web to learn more, or start with [this video](#).